

Algorithm Complexity

Space Complexity of an algorithm

- The analysis of algorithm based on **how much memory is required by an algorithm** to solve a particular problem is called space complexity of an algorithm.
- The space required by memory includes:
 - Instruction space
 - Data space
- An efficient algorithm is the one that makes the space requirement as low as possible.
- Although space complexity is important, the inexpensive memory has reduced the significance of space complexity.



Time Complexity of an algorithm

- The analysis of algorithm based on **time of computation ie**, 'time taken to execute an algorithm and get the desired results' is called *time complexity* of an algorithm.
- Main objective of time complexity is to compute the performance of different algorithms in solving same problem.
- One possible approach to measure the time complexity is to implement the algorithms using programming language and execute them.....

... but this methods have many shortcomings...like:

- i. Time is wasted in implementing all the algorithms as finally only one algorithm would be used .
- ii. Time complexity depends on number of factors as listed below those may influence the running time.
 - Type of Processor, RAM used in a machine
 - Type of Operating System used
 - Quality of code and the technique used for writing an algorithm

Change in any of the above factors result in absolute time of the algorithm. So, it is not appropriate to measure the algorithm's performance.

Some examples of finding Time Complexity of an algorithm

Example 1: A segment of algorithm involving simple Loop (with step size 1):

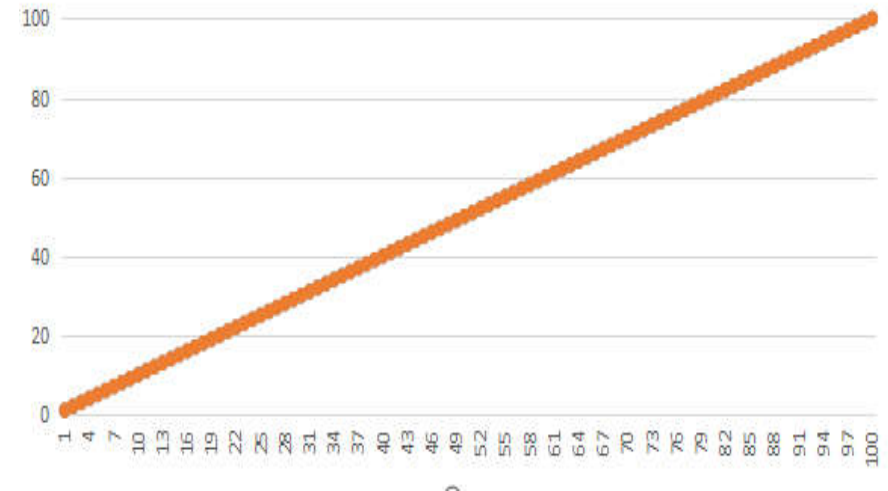
Algorithm	Equivalent code
Repeat For i = 1 to 100 by 1 Set of statements <end loop>	for(i=1; i<=100; i++) { ----statements--- }

Here, the body of the loop is repeated 100 times.

We know, the time complexity is directly proportional to the number of iterations. So, for a loop with n iterations, it is generally expressed as:

$$f(n) \propto n$$

This is an example of a Linear Loop*



***Linear Loop** is the one in which we increment or decrement a counter variable.

Finding Time Complexity of an algorithm

Example 2: A segment of algorithm involving simple Loop (with step size 2):

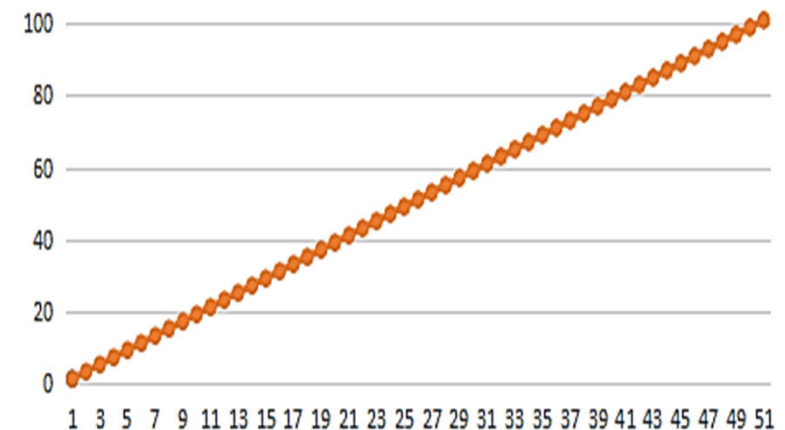
Algorithm	Equivalent code
Repeat For i = 1 to 100 by 2 Set of statements <end loop>	for(i=1; i<=100; i=i+2) { ----statements--- }

Here, the body of the loop is repeated 50 times ie **half of the value of n** because the step size here is 2 (in above example n = 100).

So in this case,

$$f(n) \propto n/2$$

Another example of Linear Loop



Finding Time Complexity of an algorithm

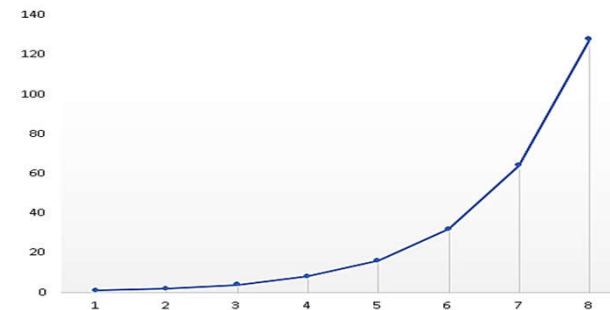
Example 3: Loop counter is **multiplied** in each iteration (eg: $i = i \times 2$)

Algorithm	Equivalent code
i=1 Repeat while i <= 100 Set of statements $i = i * 2$ <end loop>	for(i=1; i<=100; $i=i*2$) { ---statements--- }

Table: Value of i for each iteration

Iteration No.	Value of i during each iteration
1	1
2	2
3	4
4	8
5	16
6	32
7	64
8	128 (Exit Loop)

Here, the value of counter variable **i** is **not changing linearly** as it was in previous two examples.



In this example, **no. of iterations are 7 only**. The loop continues as long as following condition is met:

$$2^{\text{iterations}} \leq 100$$

Expressing this in terms of logarithms, we can say that there are

$$\log_2 100$$

ie, **7 iterations**.

So,

$$f(n) \propto \log n$$

Finding Time Complexity of an algorithm

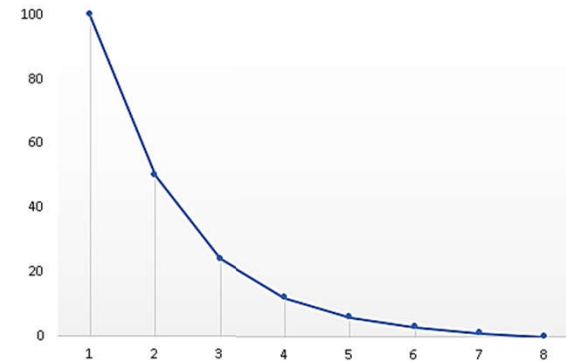
Example 4: Loop counter is divided in each iteration (eg: $i = i / 2$)

Algorithm	Equivalent code
$i=100$ Repeat while $i \geq 1$ Set of statements <u>$i = i / 2$</u> <end loop>	for($i=100$; $i \geq 1$; <u>$i=i/2$</u>) { ---statements--- }

Table: Value of i for each iteration

Iteration No.	Value of i during each iteration
1	100
2	50
3	24
4	12
5	6
6	3
7	1
8	0 (Exit Loop)

Here also, the value of counter variable i is not changing linearly.



In this example again, no. of iterations are 7 only. The loop continues as long as following condition is met:

$$100/2^{\text{iterations}} \geq 1$$

Expressing this in terms of logarithms, we can say that there are $\log_2 100$ ie, 7 iterations.

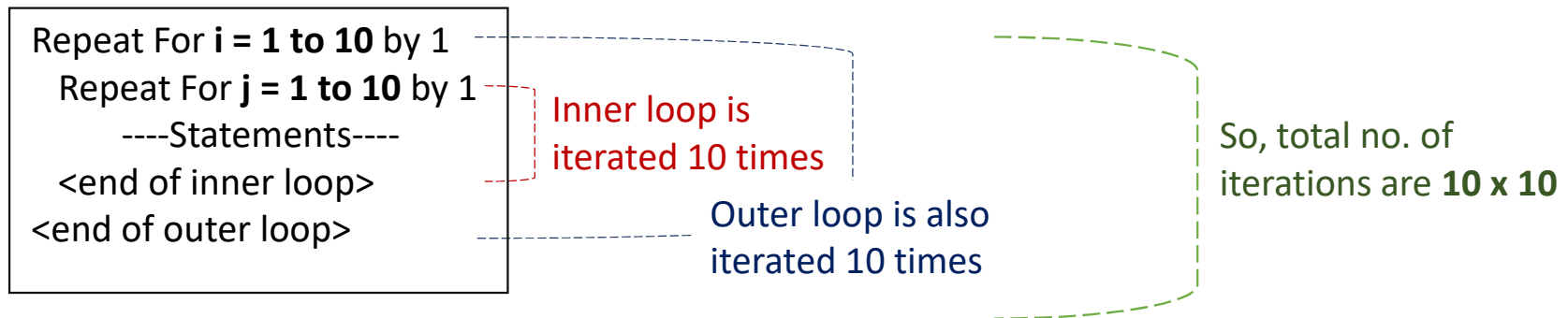
So,

$$f(n) \propto \log n$$

Finding Time Complexity of an algorithm

Example 5: In case of Nested Loops

- **Here**, we need to find out how many times each loop is iterating.
- Total number of iterations is the product of number of iterations in the inner loop and number of iterations in the outer loop.



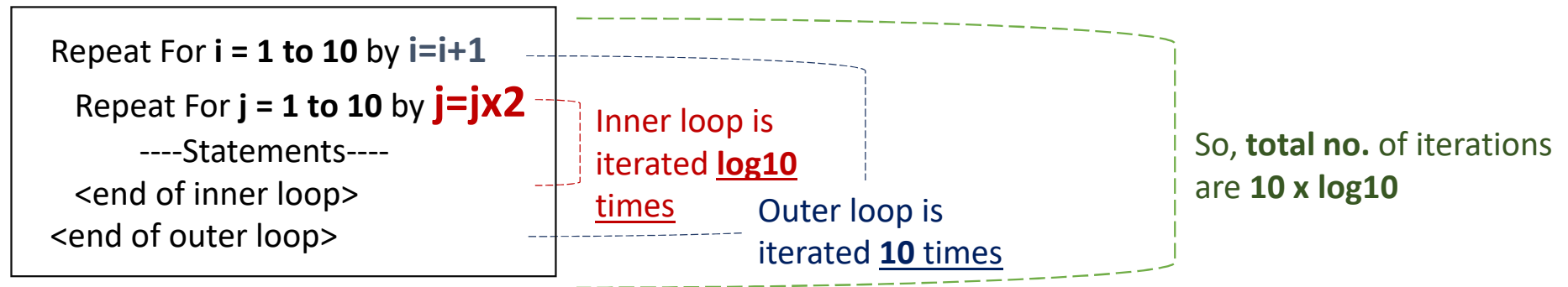
In a nutshell, if both loops iterates **n** times, then the time complexity is proportional to **n x n** ie, **n²**

So,

$$f(n) \propto n^2$$

Finding Time Complexity of an algorithm

Example 6: Another case of Nested Loops

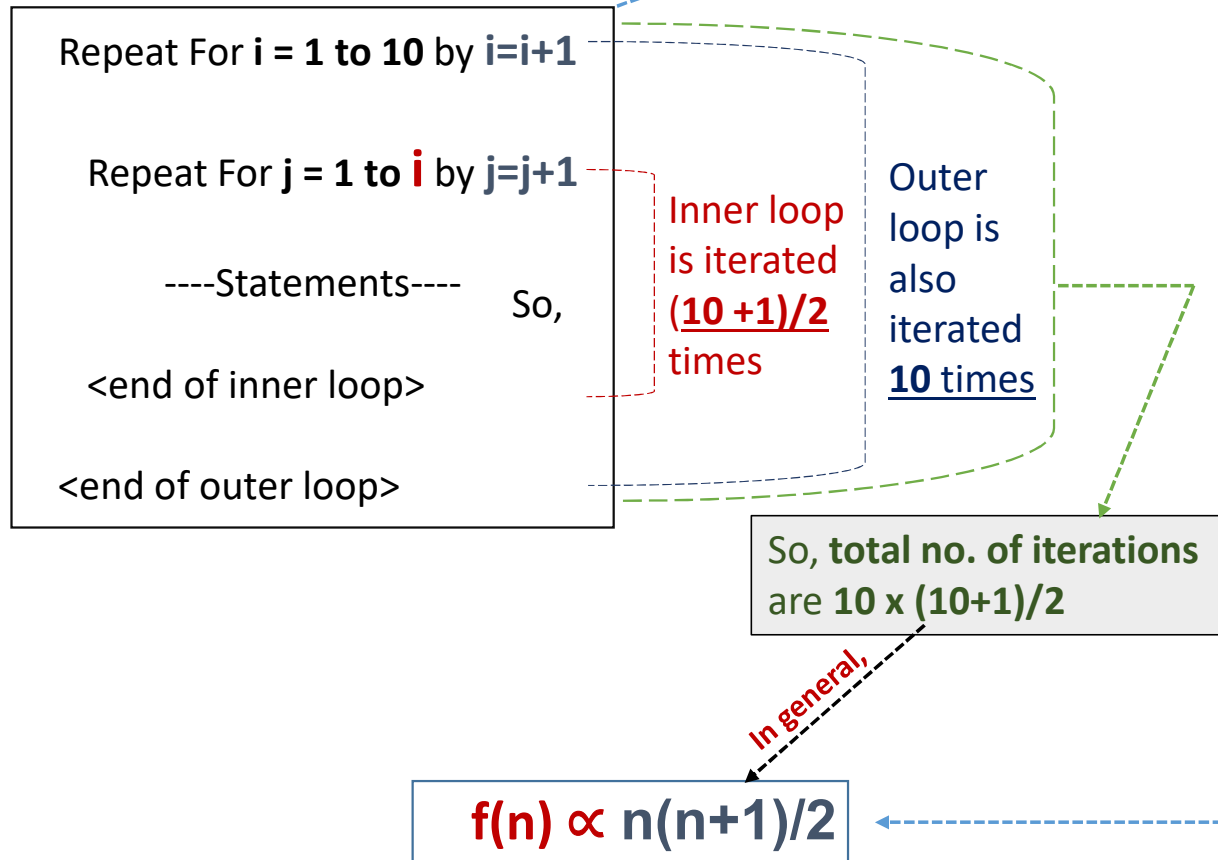


So in general,

$$f(n) \propto n \log n$$

Finding Time Complexity of an algorithm

Example 7: Another variation of Nested Loops



When value of ' i ' in the outer loop is	No. of times the body of inner loop iterated is ($j = 1$ to i)
1	1 time
2	2 times
3	3 times
4	4 times
5	5. times
6	6 times
7	7 times
8	8 times
9	9 times
10	10 times
So, total no. of times the statements inside the inner loop runs is: $1+2+3+4+5+6+7+8+9+10 = 55$	
No. of times outer loop iterated is : 10	
$\therefore$ No. of times inner loop iterated is : $55/10 = 11/2 = (10+1)/2$	